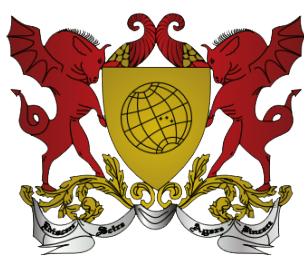


Aprendizado de Máquinas

Rodolpho Neves



Universidade Federal de Viçosa

Reitor

Demetrius David da Silva

Vice-Reitora

Rejane Nascentes



Diretor

Francisco de Assis de Carvalho Pinto

Campus Universitário, 36570-900, Viçosa/MG

Telefone: (31) 3612 1251

Layout: Ana Luísa Medeiros e Stéfany Peron

Editoração Eletrônica: Ana Luísa Medeiros

Edição de conteúdo e CopyDesk: João Batista Mota



Significado dos ícones da apostila

Para facilitar o seu estudo e a compreensão imediata do conteúdo apresentado, ao longo de todas as apostilas, você vai encontrar essas pequenas figuras ao lado do texto. Elas têm o objetivo de chamar a sua atenção para determinados trechos do conteúdo, com uma função específica, como apresentamos a seguir.



DESTAQUE: são definições, conceitos ou afirmações importantes às quais você deve estar atento.



GLOSSÁRIO: Informações pertinente ao texto, para situá-lo melhor sobre determinado termo, autor, entidade, fato ou época, que você pode desconhecer.

SAIBA MAIS: se você quiser complementar ou aprofundar o conteúdo apresentado na apostila, tem a opção de links na internet, onde pode obter vídeos, sites ou artigos relacionados ao tema.



PARA REFLETIR: vai fazer você relacionar um tópico a uma situação externa, em outro contexto



Exercícios Propostos: são momentos pra você colocar em prática o que foi aprendido



Currículo

Rodolpho Vilela Alves Neves

Possui graduação (2011) em Engenharia Elétrica pela Universidade Federal de Viçosa, mestrado (2013) e doutorado (2018) também em Engenharia Elétrica pela Universidade de São Paulo. Entre 2015 e 2016, foi pesquisador visitante na Aalborg University, Dinamarca.

Atualmente, é professor adjunto no Departamento de Engenharia Elétrica da UFV. Atua no tema de controle inteligente de sistemas dinâmicos, principalmente no controle de geração distribuída e de sistemas de energia.

Contato: rodolpho.neves@ufv.br



4. *Redes neurais artificiais*

Nas disciplinas passadas, vocês aprenderam sobre modelagem matemática e modelos estatísticos. Estas ferramentas são muito úteis quando precisamos resolver problemas estruturados, com relação direta de entrada-saída. Porém, são muito simples quando comparadas ao que o cérebro é capaz de fazer.

Estudos sobre neurologia revelaram que a unidade básica do cérebro de animais e humanos é o neurônio. Os neurônios são interconectados com outros e são capazes de transmitir sinais entre suas milhares de conexões simultaneamente. Esses estudos revelaram ainda que, quanto mais complexo é a espécie (mais inteligente), mais complexa (maior número de conexões) é a rede de neurônios dela. A relação entre a complexidade das redes neurais dos humanos e a inteligência da espécie levou cientistas a desenvolverem redes neurais artificiais para resolverem problemas complexos, assim como o cérebro humano.



Redes neurais artificiais (RNAs) são modelos matemáticos que combinam funções aritméticas para a solução de algum problema. O propósito das RNAs é imitar o comportamento cognitivo do cérebro humano a partir de dados fornecidos ao modelo e de ajustes nos parâmetros das funções.

Algumas tarefas que podem ser realizadas com o uso de RNAs são:

- Classificação de padrões;
- Predição de comportamentos; e,
- Modelagem de eventos e funções matemáticas.

É importante salientar que as RNAs não são rivais das outras ferramentas computacionais de aprendizado de máquinas. Cada ferramenta funciona melhor para estabelecer relações de comportamento em diferentes tipos de problemas. Cabe ao cientista, que está desenvolvendo o modelo, pesquisar sobre qual é a melhor arquitetura de aprendizado de máquina que obtém o melhor resultado para cada problema.

De maneira análoga aos neurônios biológicos, as RNAs têm entradas e saídas. A Figura 9 apresenta a estrutura de uma rede neural artificial, composta por três camadas: uma de entrada, uma oculta e outra de saída. A RNA pode ter várias entradas para cada neurônio, mas cada um deles apresenta apenas uma saída. Os neurônios podem ser combinados de diferentes formas para construir estruturas complexas de redes, que, por sua vez, podem ter uma ou mais camadas de neurônios escondidos. São nas camadas escondidas que acontecem as inferências e a lógica computacional das RNAs.

A teoria sobre redes neurais não é nova. Entretanto, na época em que foram desenvolvidos os primeiros algoritmos, não havia tanto poder computacional quanto hoje. Além disso, com o avanço da eletrônica, da internet e da criação de grandes bancos de dados, é possível explorar todo o potencial das RNAs e suas variantes, para resolver problemas desafiadores, como o reconhecimento de imagens e o processamento de linguagem natural.

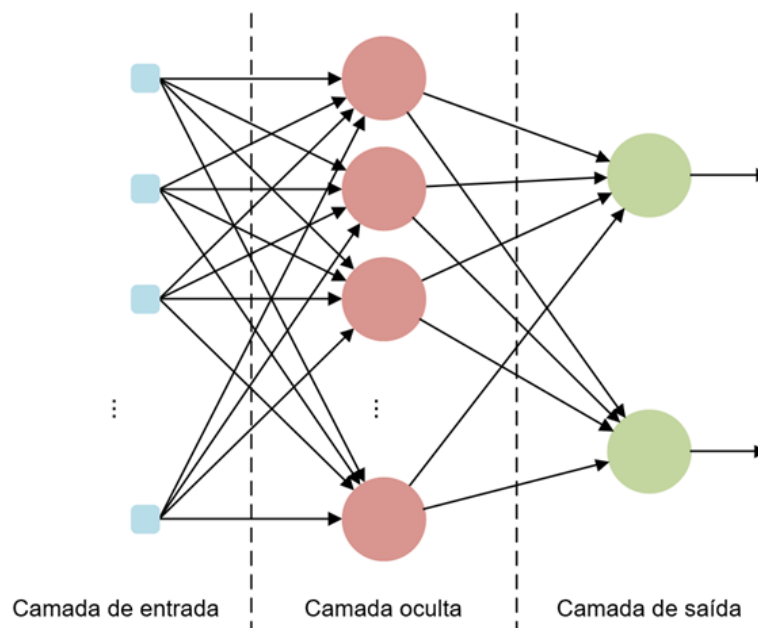


Figura 9 – Estrutura de uma rede neural artificial

1. REDES PERCEPTRON E ADALINE

Vamos iniciar a discussão sobre redes neurais artificiais a partir da estrutura mais simples, o perceptron, que é uma RNA composta por apenas um neurônio, ou seja, uma saída. Entretanto, dependendo do tipo de problema que for aplicado, ele pode ter n entradas.

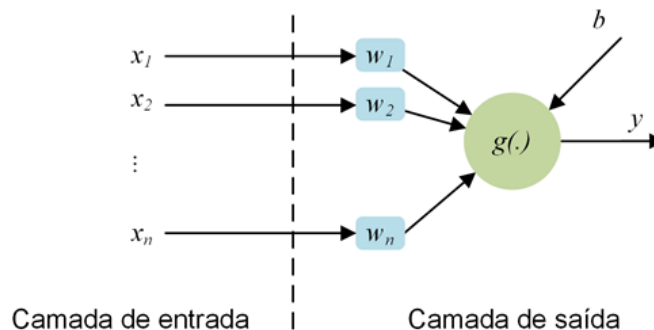


Figura 10 – Estrutura de um perceptron

A Figura 10 mostra a estrutura de um *perceptron*. Ele pode ter n entradas, representadas pelo vetor de entrada $X = [x_1, x_2, \dots, x_n]$ e uma saída, y . Cada neurônio apresenta características de pesos sinápticos e um potencial limiar de ativação, representado pelo vetor

$W = [w_1, w_2, \dots, w_n]^T$ e por b , respectivamente. Os pesos sinápticos são os atributos do perceptron que ponderam a relevância de cada uma das entradas para inferir uma saída. O potencial limiar de ativação b , do inglês *bias*, é uma constante que representa o valor mínimo que as entradas ponderadas pelos pesos sinápticos devem alcançar para que a função de ativação $g(\cdot)$ altere a saída y .

2. REPRESENTAÇÃO MATEMÁTICA DO MODELO DE REDE PERCEPTRON

O objetivo do ajuste de um *perceptron* é encontrar os valores dos pesos sinápticos e do *bias*

para que a predição da saída do neurônio seja a mais precisa possível. Como o perceptron é a forma mais simples de RNA, suas aplicações são limitadas aos problemas de classificação com classes linearmente separáveis. A Figura 11(a) apresenta um conjunto de amostras com duas classes distintas e linearmente separáveis; ou seja, que pode ser separado por uma função linear, como uma reta, um plano ou um hiperplano. A Figura 11(b) apresenta um conjunto de amostras que não pode ser separado com uma única reta.

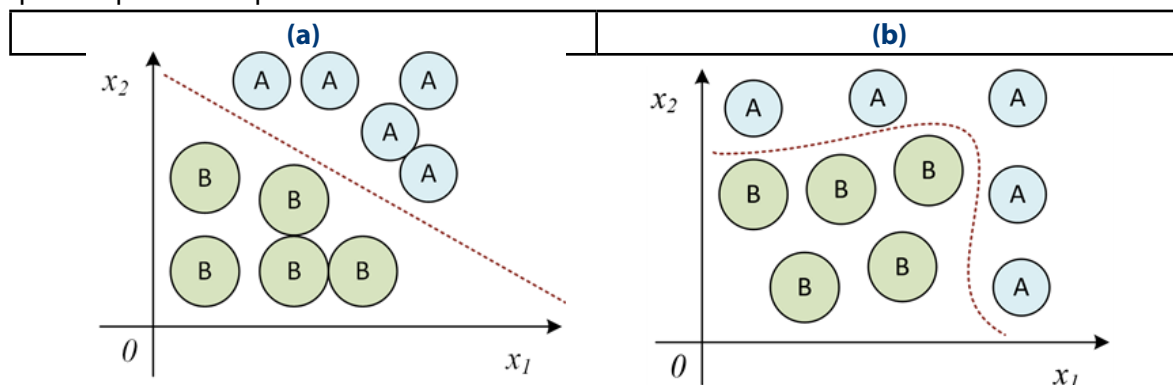


Figura 11 – Classe de amostras (a) linearmente separáveis e (b) não linearmente separáveis

Matematicamente, podemos analisar o perceptron como:

$$y(X) = g(X \cdot W + b)$$

sendo $X \cdot W$ o produto matricial dos vetores X e W . A função de ativação $g(\cdot)$ é a responsável por inferir se uma determinada amostra X pertence à classe A ou B. É possível interpretar a equação $X \cdot W + b = 0$ como retas que cortam o espaço formado por (x_1, x_2) , conforme apresentado na Figura 12. As características das retas são dadas pelos pesos sinápticos W e o potencial de ativação b .



Durante a fase de ajuste dos parâmetros, fase de treinamento da RNA, W e b serão alterados até que o *perceptron* classifique todas as amostras apresentadas de maneira correta. Com isso, várias configurações de retas podem ser alcançadas, dependendo dos parâmetros e da função de ativação utilizados. O uso de funções de ativação não lineares gera um mapeamento não linear entre as variáveis de entrada e a saída do modelo da RNA, adicionando complexidade às inferências que serão feitas pelos neurônios.

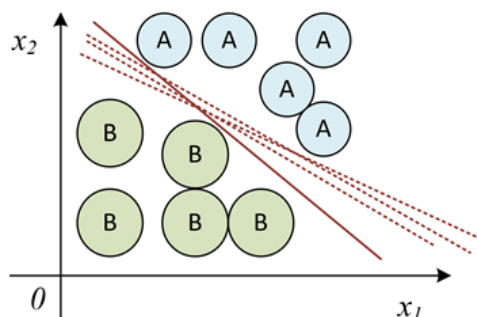


Figura 12 – Amostras linearmente separáveis com separações feitas pelo perceptron

2.1. Funções de ativação

As funções de ativação, representadas pela função $g(\cdot)$, mais utilizadas para a construção

de modelos de RNA são as sigmoide e tangente hiperbólica e suas funções particulares, degrau unitário e função sinal.

- **Sigmoide**

É uma das funções não lineares mais populares. A saída da função pode ser expressa em 0 ou 1 para valores de entrada entre $-\infty$ e $+\infty$. A sigmoide é apresentada na Figura 13 e pode ser definida matematicamente por:

$$g(x) = \text{sigm}(x) = \frac{1}{1 + e^{-\beta x}}$$

na qual x é a entrada da função e β ajusta a inclinação da sigmoide para valores de entrada próximos de zero. Quanto maior o valor de β mais inclinada é a curva entre 0 e 1.

Uma função particular da sigmoide é quando o β tende ao infinito e a função sigmoide resulta em valores discretos de 0 e 1. Esta função particular é denominada função degrau unitário e é definida como:

$$\text{degrau}(x) = \begin{cases} 1, & \text{se } x \geq 0 \\ 0, & \text{se } x < 0 \end{cases}$$

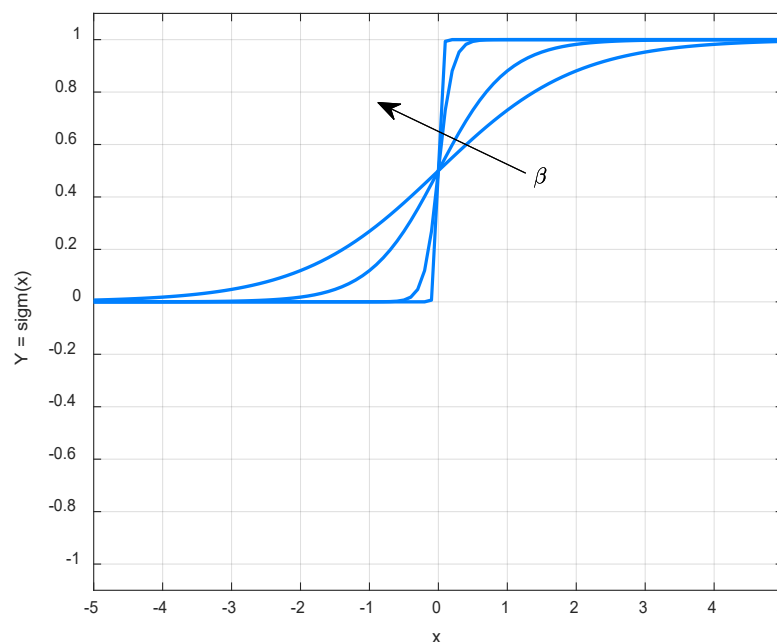


Figura 13 – Gráfico da função sigmoide

A vantagem da utilização da sigmoide, ao invés da função degrau unitário, é que ela é derivável em todo o domínio de x . Já a função degrau não é derivável em 0, o que limita a aplicação de algoritmos de treinamento, como veremos adiante. A desvantagem da função sigmoide é que ela não resulta em valores negativos. Esta desvantagem é resolvida pela função de ativação tangente hiperbólica.

- **Tangente hiperbólica**

A tangente hiperbólica, ou tanh, é muito parecida com a função sigmoide. Entretanto, a tanh é centrada em zero, o que significa que ela é simétrica em relação à origem e pode resultar em valores entre -1 e 1. A tangente hiperbólica é apresentada graficamente na Figura 14 e é definida matematicamente como:

$$g(x) = \tanh(x) = \frac{2}{1 + e^{-2\beta x}} - 1$$

Da mesma forma que na sigmoide, o termo β muda a inclinação da tangente hiperbólica. Quanto maior o valor de β , mais inclinada é a transição de -1 para 1 no domínio de x . A função particular da tangente hiperbólica, quando β é muito grande, é denominada função sinal, que é muito parecida com a função degrau unitário, mas a saída vai de -1 à 1. Matematicamente, a função sinal é definida como:

$$\text{sinal}(x) = \begin{cases} 1, & \text{se } x \geq 0 \\ -1, & \text{se } x < 0 \end{cases}$$

Assim como no caso da função degrau unitário, a função sinal(x) é não derivável em 0, sendo uma vantagem para o uso da função tangente hiperbólica em algoritmos de treinamento de RNAs que necessitam da derivação da função de ativação.

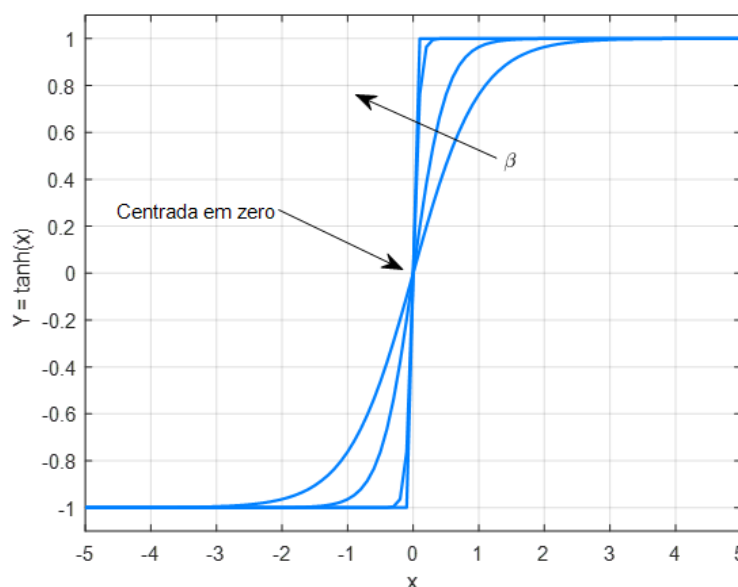


Figura 14 – Gráfico da função tangente hiperbólica (tanh)

3.TREINAMENTO E OPERAÇÃO DO PERCEPTRON

Com todos os conceitos concluídos, o próximo passo é definir o algoritmo para ajustar os parâmetros do *perceptron*, os pesos sinápticos e o limiar de ativação. A etapa de ajuste dos parâmetros é denominada treinamento da RNA, iniciada pela definição das regras de atualização dos parâmetros dos neurônios.

Seja $X^{(k)}$ o vetor de entrada da amostra k , $d^{(k)}$ o respectivo valor desejado para a saída y da amostra k , W o vetor de pesos sinápticos e b o limiar de ativação do *perceptron* que será treinado.

Os parâmetros serão ajustados enquanto houver erro na fase de treinamento do *perceptron*, pelas seguintes equações:

$$\begin{cases} W^{atual} = W^{anterior} + \eta(d^{(k)} - y)X^{(k)} \\ b^{atual} = b^{anterior} + \eta(d^{(k)} - y)(-1) \end{cases}$$

na qual η é a taxa de aprendizagem do treinamento.

A taxa de aprendizagem do treinamento (η) deve ser definida por um valor entre $0 < \eta < 1$. Essa taxa indica o quão rápido o treinamento vai acontecer e qual a taxa de atualização dos pesos sinápticos e do limiar de ativação. Valores muito altos de η podem resultar na não convergência do algoritmo e valores muito pequenos, em um custo computacional elevado para convergir. Geralmente, o valor de η varia entre 0,1% e 10%.



Uma dica de “boa prática computacional a se fazer” é considerar o limiar de ativação do perceptron um dos pesos sinápticos a serem ajustados, fazendo $W = [b, w_1, w_2, \dots, w_n]^T = [w_0, w_1, w_2, \dots, w_n]^T$, com $b = w_0$, e $X = [-1, x_1, x_2, \dots, x_n]$. Assim, ao ajustar o vetor $W^{atual} = W^{anterior} + \eta(d^{(k)} - y)X^{(k)}$, o limiar de ativação (*bias*) é ajustado ao mesmo tempo e na mesma linha do algoritmo de treinamento.

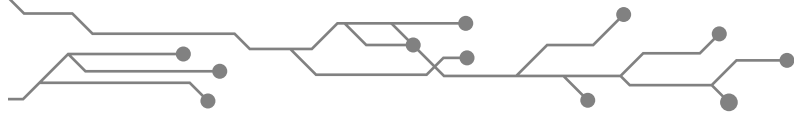
O algoritmo de treinamento do perceptron pode ser implementado como:

Início do treinamento – Perceptron:

1. Carregar o conjunto de treinamento para a matriz X;
2. Carregar o conjunto de saídas desejadas para o vetor d;
3. Concatenar a coluna de valores -1 à matriz X, representando a entrada para o bias;
4. Atribuir valores aleatórios normalizados aos pesos sinápticos W, lembrando que b está no vetor W;
5. Escolher o valor da taxa de aprendizado η ;
6. Determine o número máximo de épocas de treinamento $epoca_{max}$;
7. Iniciar o contador de épocas do treinamento ($epoca=0$);
8. Faça até que erro=0:
 - 8.1 Atribua erro=0;
 - 8.2 Para cada amostra de treinamento $\{X^{(k)}, d^{(k)}\}$
 - 8.2.1 Faça $u = X^{(k)}W$;
 - 8.2.2 Faça $y = sinal(u)$;
 - 8.2.3 Se $y \neq d^{(k)}$:
 - 8.2.3.1 Faça $W^{atual} = W^{anterior} + \eta(d^{(k)} - y)X^{(k)}$;
 - 8.2.3.2 Atribua erro=1;
 - 8.3 Faça $epoca=epoca+1$;
 - 8.4 Se $epoca > epoca_{max}$, pare o treinamento.

Fim

O treinamento do modelo pode acontecer até que o *perceptron* consiga classificar todos os elementos do banco de dados fornecido ou até que o número de épocas máximo tenha sido atingido. Caso a segunda condição tenha sido alcançada, significa que o modelo não convergiu para uma estrutura que acerte todas as entradas. Talvez seja necessário reajustar a taxa de aprendizagem ou aumentar o número de épocas de treinamento.



Depois de treinado, o *perceptron* está pronto para ser usado. O algoritmo de operação dele consiste em preparar o vetor de amostra e aplicar ao perceptron da seguinte maneira:

Início:

1. Carregar o vetor de amostra X ;
2. Concatenar o valor -1 à matriz X , representando a entrada para o bias;
3. Carregar o vetor de pesos resultante do treinamento W , lembrando que b está no vetor W ;
4. Faça $u = X^{(k)} \cdot W$;
5. Faça $y = \text{sinal}(u)$;
6. Se $y > 0$:
 - 6.1 A amostra X pertence à classe A;
7. Caso contrário:
 - 7.1 A amostra X pertence à classe B;

Fim

Perceba que durante a fase de treinamento, o erro utilizado para ajustar os parâmetros do *perceptron* era a diferença entre o valor desejado e o resultado da função sinal, $d^{(k)} - y$. Como $y = \text{sinal}(u)$, os valores que poderiam resultar de y era apenas -1 e 1. Este cálculo de erro não consegue fazer um ajuste fino no posicionamento do hiperplano que separa as classes de amostras, sendo necessário outra regra de atualização dos pesos sinápticos e do limiar de ativação.

- **Exemplo**

Uma das aplicações que motivou o desenvolvimento das redes perceptron foi a interpretação lógica de sinais. A modelagem de uma operação lógica OU ou E de forma automática poderia criar uma ferramenta muito útil para a engenharia e tecnologia da época.

A operação lógica OU funciona da seguinte maneira: dado um conjunto de entradas digitais $X = [x_1, x_2]$, a lógica OU entre elas obedece a seguinte tabela verdade:

0	0	0
0	1	1
1	0	1
1	1	1

Isto significa que, dado um valor verdadeiro em qualquer uma das entradas, é atribuído um valor verdadeiro à saída. Como esta função poderia ser modelada por uma rede perceptron?

4. REGRA DELTA GENERALIZADA

A atualização da regra de ajuste dos parâmetros do perceptron veio junto com o desenvolvimento do elemento linear adaptativo (do inglês *adaptive linear elemento - Adaline*), que foi

uma melhora na estrutura do perceptron. Ele utiliza o erro entre o valor desejado do banco de dados e a equação do hiperplano do perceptron. A Figura 15 apresenta a estrutura do Adaline e a síntese do erro para ajuste dos parâmetros durante a fase de treinamento. Repare que, nesta estrutura, o limiar de ativação foi associado ao peso do perceptron, como mencionado na dica de boa prática de programação.

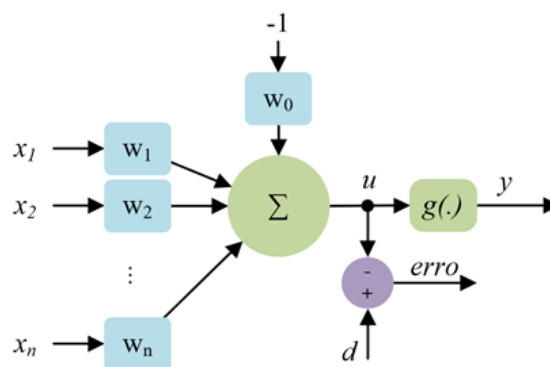


Figura 15 – Estrutura do Adaline



A partir da utilização do sinal u para calcular $erro = d - u$, o ajuste fino dos parâmetros é realizado e o algoritmo de treinamento garante que os parâmetros do hiperplano ótimo, que separa as amostras de um banco de dados, serão selecionados. Assim, independentemente dos valores iniciais aleatórios sorteados para o vetor de pesos sinápticos e para o limiar de ativação, o algoritmo de treinamento vai convergir para o conjunto de parâmetros ótimo do hiperplano. Esta técnica é denominada Regra Delta, mas também é conhecida como mínimo erro médio quadrado (do inglês *least mean square error: LMS*) ou, ainda, gradiente descendente.

O LMS é calculado a partir da soma dos erros, obtidos por $erro = d - u$ ao quadrado e divididos por dois. Matematicamente, o LMS pode ser calculado por:

$$E(W) = \frac{1}{2} \sum_{k=1}^p (d^{(k)} - u)^2 = \frac{1}{2} \sum_{k=1}^p (d^{(k)} - X^{(k)}W)^2$$

na qual $E(W)$ é o erro médio quadrático em função do vetor de pesos W e p é o número total de amostras no banco de dados de treinamento do modelo. Como o objetivo é minimizar o erro $E(W)$, devemos ajustar os valores dos pesos sinápticos a partir do gradiente do erro e encontrar o ponto de erro mínimo para um vetor de pesos sinápticos ótimo. Então, o gradiente do erro médio quadrático, que é a derivada do erro em função do vetor de pesos sinápticos, pode ser calculado por:

$$\nabla E(W) = \frac{\partial E(W)}{\partial W} = - \sum_{k=1}^p (d^{(k)} - u) X^{(k)}$$

Sendo assim, o ajuste dos erros a partir do erro médio quadrático (Regra Delta) pode ser feito por:

$$\Delta W = -\eta \nabla E(W) = \eta \sum_{k=1}^p (d^{(k)} - u) X^{(k)}$$

considerando o sinal negativo na taxa de atualização dos pesos, porque a atualização dos pesos deve ser no sentido oposto ao do gradiente. Reescrevendo a Regra Delta em função dos pesos atual e anterior, tem-se:

$$\mathbf{W}^{\text{atual}} = \mathbf{W}^{\text{anterior}} + \eta \sum_{k=1}^p (\mathbf{d}^{(k)} - \mathbf{u}) \mathbf{X}^{(k)}$$

lembrando que o potencial de ativação está inserido no vetor $\mathbf{W}$ e a entrada -1 está inserida no conjunto de amostras $\mathbf{X}^{(k)}$. Iterativamente, o erro médio quadrático vai diminuindo até encontrar o ponto de erro médio quadrático mínimo e o vetor de pesos ótimo. A Figura 16 apresenta uma curva de erro médio quadrático em função dos ajustes do vetor de peso, partindo do valor inicial dos pesos $\mathbf{W}^{(0)}$ até o último valor $\mathbf{W}^{\text{atual}}$.

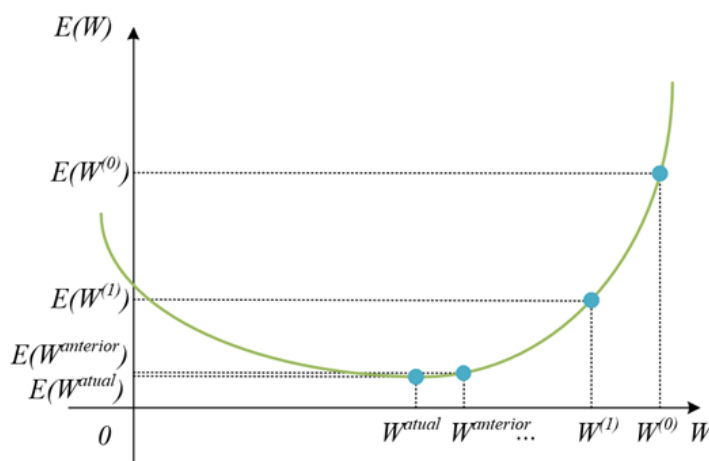


Figura 16 – Análise gráfica do erro médio quadrático

Uma forma de verificar se o vetor de pesos atingiu o valor ótimo (ou não está mais sofrendo alterações) é calcular o erro quadrático médio obtido pelo modelo na fase de treinamento. O erro quadrático médio pode ser calculado por:

$$\text{EQM}(\mathbf{W}) = \frac{1}{p} \sum_{k=1}^p (\mathbf{d}^{(k)} - \mathbf{u})^2$$

Os parâmetros do hiperplano não estão sendo mais alterados - ou estão sendo muito pouco alterados - quando a diferença entre duas épocas consecutivas de $\text{EQM}(\mathbf{W})$ é menor do que um limiar muito pequeno. Ou seja, quando:

$$|\text{EQM}(\mathbf{W}^{\text{atual}}) - \text{EQM}(\mathbf{W}^{\text{anterior}})| \leq \varepsilon$$

sendo ε o limiar de precisão, da ordem de 10^{-5} , pode-se considerar que o ponto de mínimo do erro médio quadrático foi encontrado. O modelo está treinado quando esta combinação é alcançada ou quando o número de épocas de treinamento tenha atingido o valor de época máximo.

O algoritmo de treinamento do Adaline é muito parecido com o do perceptron. A diferença entre eles é a utilização da regra Delta para o ajuste dos parâmetros do hiperplano de separação das classes, o vetor de pesos sináptico e o limiar de ativação. O pseudocódigo do algoritmo é:

Início do treinamento – Adaline (com regra Delta):

1. Carregar o conjunto de treinamento para a matriz $\mathbf{X}$ com a entrada do limiar -1;
2. Carregar o conjunto de saídas desejadas para o vetor $\mathbf{d}$;

3. Atribuir valores aleatórios normalizados aos pesos sinápticos W (com o w_0);
4. Escolher o valor da taxa de aprendizado η e para o limar de precisão ϵ ;
5. Determinar o número máximo de épocas de treinamento $epoca_{max}$;
6. Iniciar o contador de épocas do treinamento ($epoca=0$);
7. Faça até que $|\mathbf{EQM}(\mathbf{W}^{atual}) - \mathbf{EQM}(\mathbf{W}^{anterior})| \leq \epsilon$:
 - 7.1 Faça $\mathbf{EQM}(\mathbf{W}^{anterior}) = \mathbf{EQM}(\mathbf{W}^{atual})$;
 - 7.2 Faça $\mathbf{EQM}(\mathbf{W}^{atual}) = 0$;
 - 7.3 Para cada amostra de treinamento $\{X^{(k)}, d^{(k)}\}$, faça:
 - 7.3.1 $u = X^{(k)}W$;
 - 7.3.2 $\mathbf{EQM}(\mathbf{W}^{atual}) = \mathbf{EQM}(\mathbf{W}^{atual}) + (d^{(k)} - u)^2$;
 - 7.3.3 $\mathbf{W}^{atual} = \mathbf{W}^{anterior} + \eta(d^{(k)} - u)X^{(k)}$;
 - 7.4 Faça $\mathbf{EQM}(\mathbf{W}^{atual}) = \mathbf{EQM}(\mathbf{W}^{atual})/p$;
 - 7.5 Faça $epoca=epoca+1$;
 - 7.6 Se $epoca > epoca_{max}$, pare o treinamento.

Fim

O algoritmo da fase de operação do Adaline é idêntico ao do perceptron. Entretanto, depois de finalizar a fase de treinamento do Adaline, os parâmetros ótimos do hiperplano de separação das classes fazem com que a divisão seja equidistante das fronteiras de separação das classes (Figura 17). Em outras palavras, independentemente do valor inicial que foi dado aos pesos sinápticos e ao limiar de ativação do neurônio, os parâmetros vão convergir para o valor ótimo do vetor de pesos.

Essa é uma vantagem muito significativa ao aplicar a regra Delta para a busca paramétrica do hiperplano. Porém, o Adaline ainda não é capaz de resolver o problema das classes não linearmente separáveis, como mostrado na Figura 11(b). A separação destas classes vai ser feita com a utilização de vários perceptron em camadas, também conhecido como a rede perceptron multicamadas (PMC).

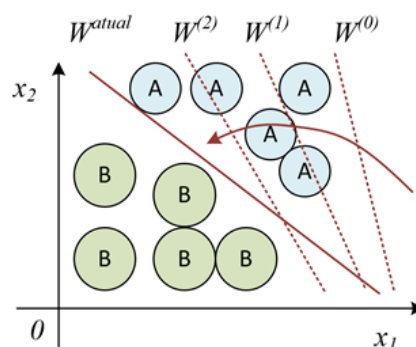


Figura 17 – Amostras linearmente separáveis com separações feitas pelo Adaline

5. REDES PERCEPTRON MULTICAMADAS

As redes neurais artificiais são denominadas redes de alimentação por etapas a frente, do inglês *feed-forward neural network*. As redes *feed-forward* têm esse nome porque a informação que cada camada da rede recebe vem da alimentação fornecida pela camada anterior. Outra definição para rede neural feed-forward é que não há realimentação da informação no modelo.

Redes perceptron multicamadas (PMC) são exemplos de redes neurais *feed-forward*. A PMC é composta por uma camada de entrada, camadas de neurônios ocultas e uma camada de saída (Figura 9). Os neurônios na camada escondida são responsáveis por processar e extrair informações dos dados de entrada para que eles sejam sintetizados pela camada de saída. Outro exemplo que pode ser analisado de uma rede PMC é apresentada na Figura 18.

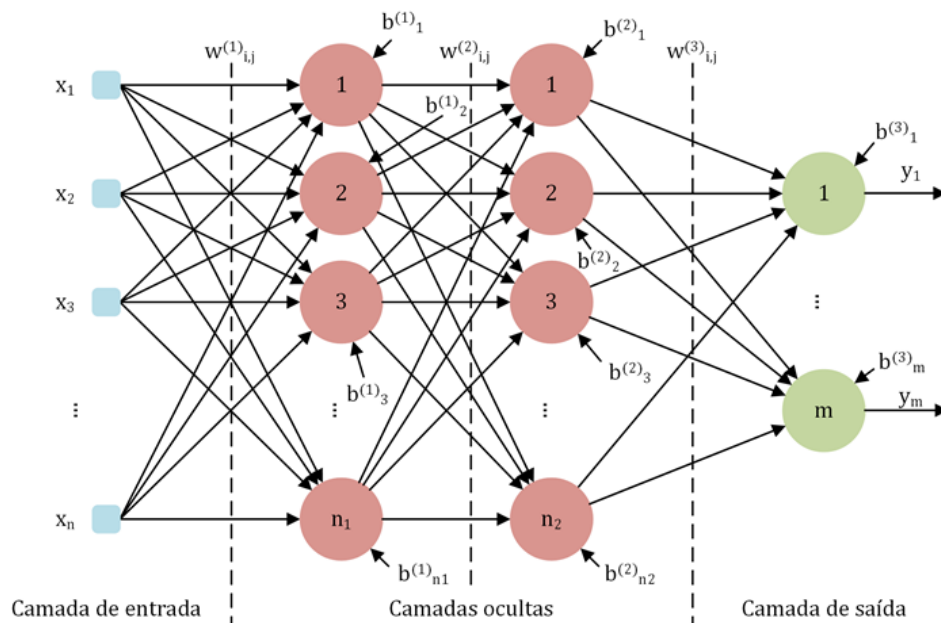


Figura 18 – Estrutura de uma rede perceptron multicamadas

A rede PMC da Figura 18 apresenta uma camada de entrada, com η entradas, duas camadas ocultas com n_1 e n_2 neurônios por camadas, respectivamente, e uma camada de saída com m neurônios.



Na configuração de rede *feed-forward*, todas as entradas da primeira camada estão conectadas aos neurônios primeira camada oculta e todos os neurônios da primeira camada oculta estão conectados aos neurônios da segunda camada oculta. Por sua vez, a segunda camada oculta também está toda conectada à camada de saída. A cada um dos neurônios estão associados pesos sinápticos que ponderam cada uma de suas entradas e um limiar de ativação. A matriz de pesos sinápticos de cada camada é representada por $W_{i,j}^{(L)}$, na qual L é a camada, i é o número da entrada e j é o número do neurônio na camada e o limiar de ativação de cada neurônio é dado por $b_j^{(L)}$.

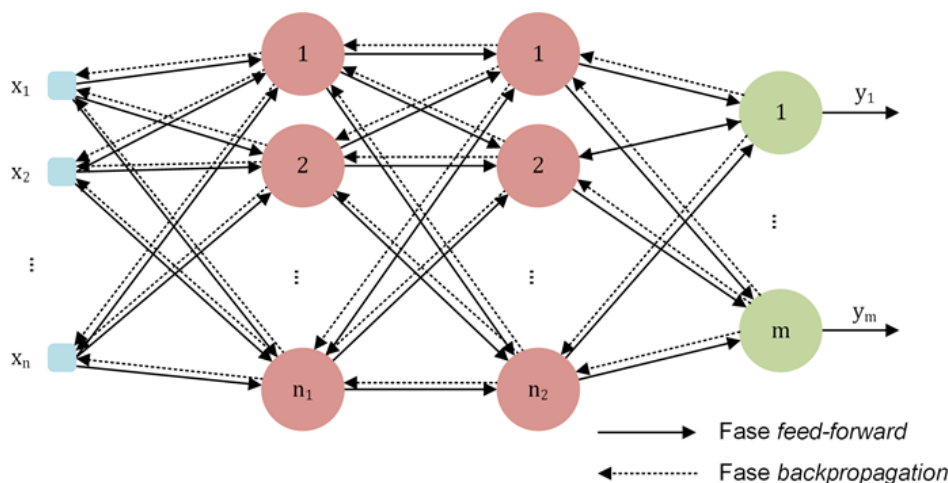


Figura 19 – Funcionamento das fases *feed-forward* e *backpropagation* no treinamento da rede PMC

O funcionamento da rede perceptron multicamadas é baseado no funcionamento do neurônio perceptron, mas ocorre de forma simultânea para todos os neurônios na camada. As entradas são repassadas para a primeira camada oculta, que faz a soma ponderada pelos pesos sinápticos e compara ao limiar de ativação. A soma das entradas ponderadas ao limiar de ativação serve de entrada para a função de ativação que fornece seu resultado como entrada para os neurônios da segunda camada oculta. O processo se repete na segunda camada oculta que fornece as entradas para a camada de saída. Na camada de saída, as entradas são processadas da mesma forma e o resultado é a predição da rede PMC.

6. ALGORITMO BACKPROPAGATION

As matrizes de pesos sinápticos e limiares de ativação são ajustadas na fase de treinamento da rede perceptron multicamadas. Os ajustes desses parâmetros são feitos pelo método de propagação reversa, do inglês *backpropagation*. Este é um dos métodos mais comuns de ajuste dos parâmetros da rede PMC e sua ideia principal é que o erro entre a saída desejada e o valor inferido pela rede PMC ecoe para as camadas da rede e ajuste cada um dos parâmetros. A Figura 19 ilustra o funcionamento das fases *feed-forward* e *backpropagation* durante o processo de treinamento da rede PMC.

O erro calculado entre a saída desejada e o valor inferido pela PMC é composto por contribuições de cada um dos neurônios, por meio dos pesos sinápticos e dos limiares de ativação. Durante a fase de treinamento da rede PMC, a cada amostra apresentada para a rede, a fase *feed-forward* é realizada. Uma vez inferida o resultado para aquela amostra, o erro é calculado e os parâmetros dos neurônios são ajustados com base no gradiente descendente.

Vamos tomar como exemplo uma rede PMC com duas entradas, duas camadas ocultas e uma saída. Na primeira camada oculta, existem três neurônios, e na segunda, dois. A Figura 20 apresenta a estrutura da rede PMC descrita.

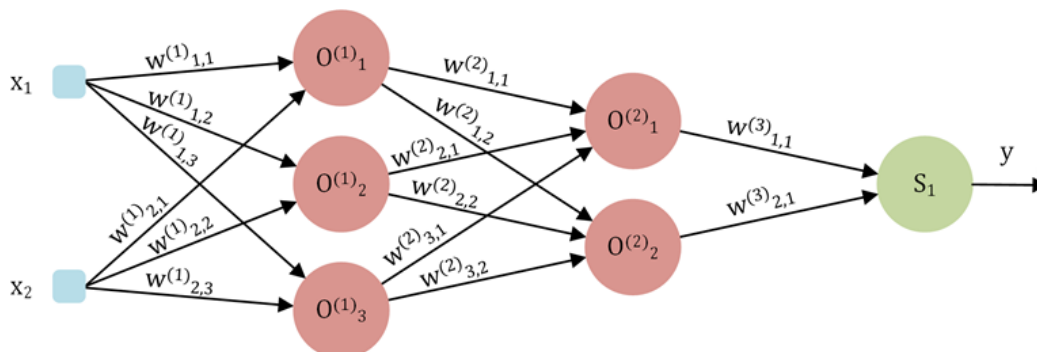
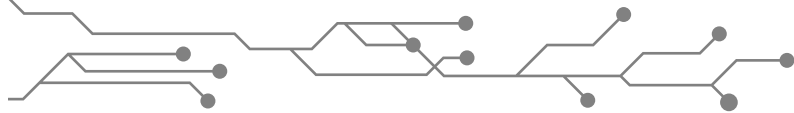


Figura 20 – Rede PMC para dedução da lei de atualização dos pesos sinápticos durante a fase *backpropagation*



A camada de entrada é composta pela entrada $X = [x_1, x_2]$. O termo relativo ao *bias* de cada neurônio foi suprimido por efeito de simplificação, mas o raciocínio pode ser extrapolado para o caso com cada limiar de ativação. Seja $I_j^{(1)}$ soma das entradas ponderada pelos pesos $w_{i,j}^{(1)}$ de cada entrada x_i , para cada neurônio j na camada 1, tal que $I_j^{(1)} = XW_j^{(1)}$. Por exemplo, para o neurônio $O_1^{(1)}$, tem-se:

$$I_1^{(1)} = XW_j^{(1)} = [x_1, x_2] \begin{bmatrix} w_{1,1}^{(1)} \\ w_{2,1}^{(1)} \end{bmatrix} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)}$$

A saída dos neurônios da primeira camada oculta é dada pela aplicação da função de ativação às entradas ponderadas pelos pesos sinápticos, resultando em $y_j^{(1)} = g(I_j^{(1)})$. A equação da saída dos neurônios pode ser generalizada como $y_j^{(L)} = g(I_j^{(L)})$, na qual $I_j^{(L)}$ é dado para cada neurônio, sendo $I_j^{(L)} = Y^{(L-1)}W_j^{(L)}$ as conexões do neurônio j da camada L com as saídas dos neurônios da camada anterior ($Y^{(L-1)}$) ponderadas pelos pesos sinápticos das conexões $W_j^{(L)}$ deste neurônio. Por exemplo, na segunda camada oculta, para o neurônio $O_2^{(2)}$, tem-se:

$$I_2^{(2)} = Y^{(L-1)}W_j^{(L)} = [y_1^{(1)}, y_2^{(1)}, y_3^{(1)}] \begin{bmatrix} w_{1,2}^{(2)} \\ w_{2,2}^{(2)} \\ w_{3,2}^{(2)} \end{bmatrix} = y_1^{(1)} w_{1,2}^{(2)} + y_2^{(1)} w_{2,2}^{(2)} + y_3^{(1)} w_{3,2}^{(2)}$$

e a saída deste neurônio é dada por $y_2^{(2)} = g(I_2^{(2)})$. De forma semelhante, a saída do neurônio de saída é dada por:

$$y_1^{(3)} = g(I_1^{(3)}) = g(y_1^{(2)} w_{1,1}^{(3)} + y_2^{(2)} w_{2,1}^{(3)})$$

A partir do cálculo da saída da rede PMC, é possível calcular o erro quadrático médio de predição para cada uma das amostras. Matematicamente, o erro médio quadrático para m neurônios, na última camada Lm de saída, é definido por:

$$E(k) = \frac{1}{2} \sum_{j=1}^m (d_j(k) - y_j^{(Lm)})^2$$

na qual $d_j(k)$ é a saída desejada para a amostra k do neurônio j . O erro médio considerando uma quantidade p de amostras é dado por:

$$EQM = \frac{1}{p} \sum_{k=1}^p E(k)$$



O algoritmo *backpropagation* utiliza a direção oposta do gradiente de *EQM* para buscar os melhores pesos $W_j^{(L)}$ com a intenção de que a rede PMC predite saídas $y_j^{(Lm)}$ mais próximas possíveis de $d_j(k)$. As atualizações dos pesos das camadas da rede PMC é feita por etapas, começando da última camada e finalizando na primeira.

A atualização dos pesos da última camada $W_j^{(Lm)}$ é feita pela regra:

$$\Delta W_j^{(Lm)} = W_j^{(Lm)}(t+1) - W_j^{(Lm)}(t) = -\eta \frac{\partial EQM}{\partial W_j^{(Lm)}} = -\eta \left(-(d_j - y_j^{(Lm)}) g'(I_j^{(Lm)}) y_j^{(Lm-1)} \right)$$

na qual $g'(I_j^{(Lm)})$ é a derivada da função de ativação, aplicada no ponto $I_j^{(Lm)}$. A regra de atualização dos pesos sinápticos da saída pode ser simplificada para:

$$W_j^{(Lm)}(t+1) = W_j^{(Lm)}(t) + \eta \delta_j^{(Lm)} y_j^{(Lm-1)}$$

com $\delta_j^{(Lm)} = (d_j - y_j^{(Lm)}) g'(I_j^{(Lm)})$. O termo $\delta_j^{(Lm)}$ é responsável por propagar o EQM para as outras camadas internas da rede PMC. A atualização dos neurônios das camadas ocultas da rede PMC é feita pela regra

$$W_j^{(L)}(t+1) = W_j^{(L)}(t) + \eta \delta_j^{(L)} y_j^{(L-1)}$$

na qual $\delta_j^{(L)} = \delta_j^{(Lm)} W_j^{(Lm)} g'(I_j^{(L)})$ é a parcela de propagação do erro de predição da rede PMC. É possível notar que a atualização dos pesos da camada (L) tem participação das saídas dos neurônios da camada $(L-1)$. Então, podemos prever que a atualização dos pesos sinápticos dos neurônios da primeira camada oculta terá participação das entradas da rede PMC.

A regra de atualização dos pesos sinápticos da primeira camada oculta, que faz interface com a camada de entrada, é dada por

$$W_j^{(1)}(t+1) = W_j^{(1)}(t) + \eta \delta_j^{(1)} x_i$$

na qual $\delta_j^{(1)} = \delta_j^{(2)} W_j^{(2)} g'(I_j^{(1)})$. Portanto, no algoritmo *backpropagation*, o gradiente descendente propaga o erro da predição para todas as camadas internas da rede PMC e é ponderado pelas entradas que são apresentadas a cada neurônio durante a fase de *feed-forward*.

A implementação de um algoritmo de treinamento de uma rede perceptron multicamada consiste em realizar a etapa *feed-forward* para prever as saídas e, em seguida, realizar a etapa *backward* para ajustar os parâmetros dos pesos sinápticos. Em termos de pseudocódigo, o algoritmo de treinamento é:

Início – Fase de treinamento – Rede Perceptron Multicamada:

1. Carregar o conjunto de treinamento para a matriz X com a entrada do limiar -1;
2. Carregar o conjunto de saídas desejadas para o vetor d ;
3. Atribuir valores aleatórios e normalizados aos pesos sinápticos W (com o w_0) de todas as camadas da rede;
4. Escolher o valor da taxa de aprendizado η e para o limiar de precisão ε ;
5. Determine o número máximo de épocas de treinamento $epoca_{max}$;

6. Iniciar o contador de épocas do treinamento ($epoca = 0$) ;
7. Faça até que $|EQM^{atual} - EQM^{anterior}| \leq \varepsilon$:
 - 7.1 Faça $EQM^{anterior} = EQM^{atual}$;
 - 7.2 Para todas as amostras de treinamento $\{X^{(k)}, d^{(k)}\}$, faça:
 - 7.2.1 Calcular o valor de $I_j^{(1)}$ e $y_j^{(1)}$ para a primeira camada;
 - 7.2.2 Calcular o valor de $I_j^{(L)}$ e $y_j^{(L)}$ para todas as camadas intermediárias;
 - 7.2.3 Calcular o valor de $I_j^{(Lm)}$ e $y_j^{(Lm)}$ para a última camada;
 - 7.2.4 Calcular o valor de $\delta_j^{(Lm)}$ para a última camada;
 - 7.2.5 Atualizar o valor de $W_j^{(Lm)}$ para a última camada;
 - 7.2.6 Calcular o valor de $\delta_j^{(L)}$ e $W_j^{(L)}$ atualizar para as camadas intermediárias;
 - 7.2.7 Calcular o valor de $\delta_j^{(1)}$ para a última camada;
 - 7.2.8 Atualizar o valor de $W_j^{(Lm)}$ para a última camada;
 - 7.3 Calcular o valor de $y_j^{(Lm)}$;
 - 7.4 Calcular o erro EQM^{atual}
 - 7.5 Faça $epoca = epoca + 1$;
 - 7.6 Se $epoca > epoca_{max}$, pare o treinamento.

Fim

Os critérios de parada do algoritmo de treinamento seguem os mesmos adotados no Adaline, variação do erro quadrático médio menor que o limiar de precisão ($|EQM^{atual} - EQM^{anterior}| \leq \varepsilon$) e o número de épocas de treinamento ultrapassar o número de épocas máximo.

A operação da rede PMC, simples como a operação do perceptron, é executada somente a fase *forward* da etapa de treinamento. O algoritmo da fase de operação, em forma de pseudocódigo é:

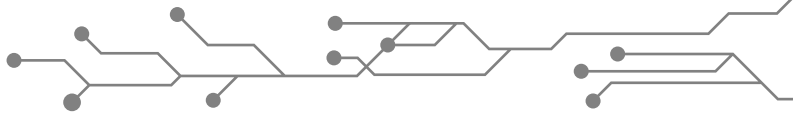
Início – Fase de operação – Rede Perceptron Multicamada:

1. Carregar o vetor de amostra X ;
2. Concatenar o valor -1 à matriz X , representando a entrada para o *bias*;
3. Carregar o vetor de pesos resultante do treinamento W de todas os neurônios e todas as camadas;
4. Calcular o valor de $I_j^{(1)}$ e $y_j^{(1)}$ para a primeira camada;
5. Calcular o valor de $I_j^{(L)}$ e $y_j^{(L)}$ para todas as camadas intermediárias;
6. Calcular o valor de $I_j^{(Lm)}$ e $y_j^{(Lm)}$ para a última camada;
7. Disponibilizar a saída da rede de acordo com os valores de $y_j^{(Lm)}$;

Fim



Como pôde ser observado, o método do *backpropagation* realiza os ajustes dos pesos sinápticos e dos limiares buscando os parâmetros ótimos que definam os hiperplanos de separação de amostras. Entretanto, dependendo do tamanho do banco de dados de treinamento e da complexidade da rede, o método de treinamento utilizando o *backpropagation* pode ser lento e computacionalmente custoso.



Desde a primeira publicação de resultados utilizando o *backpropagation*, muitos pesquisadores desenvolveram maneiras de aperfeiçoar o algoritmo de busca pelos melhores parâmetros na fase de treinamentos de redes perceptron multicamadas. Alguns algoritmos que desempenham melhor o papel de busca paramétrica são:

- **Gradiente descendente com *momentum*:** um termo de *momentum* (inércia) é adicionado à regra de atualização dos pesos e faz com que a convergência do gradiente descendente com *momentum* seja mais rápida do que sem a inércia. A regra de atualização das matrizes de pesos das camadas da PMC resulta em:

$$W_j^{(L)}(t+1) = W_j^{(L)}(t) + \alpha \left(W_j^{(L)}(t) - W_j^{(L)}(t-1) \right) + \eta \left(\delta_j^{(L)} Y_j^{(L-1)} \right)$$

na qual α é o termo do *momentum*. O valor de α pode ser escolhido entre 0 e 1, sendo que $\alpha = 0$ resulta na equação de atualização dos pesos do gradiente descendente. No início da fase de treinamento, quando a variação nos valores de $W_j^{(L)}$ é grande, o termo de *momentum* $\alpha \left(W_j^{(L)}(t) - W_j^{(L)}(t-1) \right)$ resulta em valores grandes. No final da fase de treinamento, a variação nos valores de $W_j^{(L)}$ são pequenas, o que leva o termo de *momentum* a causar variações muito pequenas.

- **Método de Levenberg-Marquardt:** é um dos métodos de ajuste de parâmetros mais aplicados em treinamento de redes PMC. Utiliza o gradiente de segunda ordem, um método baseado nos mínimos quadrados para modelagem de sistemas não lineares, que melhora a eficiência do algoritmo de treinamento *backpropagation*. A principal ferramenta deste método é a utilização da matriz Jacobiana do erro quadrático médio em função de cada um dos parâmetros que está sendo ajustado. Este método é capaz de finalizar o treinamento de uma rede PMC até 100 vezes mais rápido do que o gradiente descendente sem *momentum*.

7. ASPECTOS SOBRE TREINAMENTO DE REDES MLP

A construção de um modelo, baseado em uma rede perceptron multicamadas, requer alguns cuidados na preparação e execução do treinamento. Eles estão relacionados ao banco de dados, à capacidade de generalização do modelo e ao teste de desempenho real do modelo.

Começando pelo conjunto de dados que será utilizado, uma boa prática para criar modelos baseados em redes neurais é ter uma quantidade substancial de dados que descreva bem o fenômeno que está sendo modelado. Caso a quantidade de dados seja abastarda, selecionar quais dados são realmente necessários diminui o tempo de treinamento e pode alcançar resultados semelhantes, ou melhores, de desempenho do modelo. Feito isso, os dados devem ser preparados para o treinamento, como vimos anteriormente na Seção de “Preparação de dados”. Com os dados preparados, podemos dividir todo o banco de dados em dois conjuntos distintos: o de treinamento e o de teste.

A divisão do banco de dados em conjunto de treinamento e teste pode ser feita de maneira aleatória, utilizando de 70% a 90% do banco de dados para treinamento e de 10% a 30% para teste. Na fase de treinamento, o modelo é ajustado apenas com os dados de treinamento, restando aos dados de teste a função de verificar o comportamento do ajuste para dados que não foram apresentados anteriormente. A escolha de quais dados vão para cada conjunto pode ser feita de forma aleatória ou sistemática.

O sorteio aleatório dos dados para compor os conjuntos de treinamento e teste não garante que todos os dados serão sorteados para cada uma das etapas. Mesmo que o treinamento seja feito inúmeras vezes, existe um fator de aleatoriedade que não garante essa participação. Assim,

seria necessário um número elevado de tentativas para que os dados das fases de treinamento e de teste fossem combinados em um conjunto que possibilitasse a melhor extração de informações dos dados para o modelo. Outra maneira mais operacional de fazer a escolha desses dados é utilizando a validação cruzada de k-partições (do inglês *k-fold cross-validation*).



O método de validação cruzada *k-fold* é realizado a partir da divisão das amostras do banco de dados nos conjuntos de treinamento e de teste. Nesse método é escolhido o número de partições (k) que serão feitas no banco de dados. O número k de partições também indica quantos treinamentos serão realizados com o banco de dados particionado (Figura 21). Nesse caso, foi escolhido o valor $k = 5$, totalizando 5 treinamentos, para que todas as amostras participem como conjunto de treinamento e conjunto de teste, e um percentual de 20% de dados para o conjunto de teste e 80% para o de treinamento.

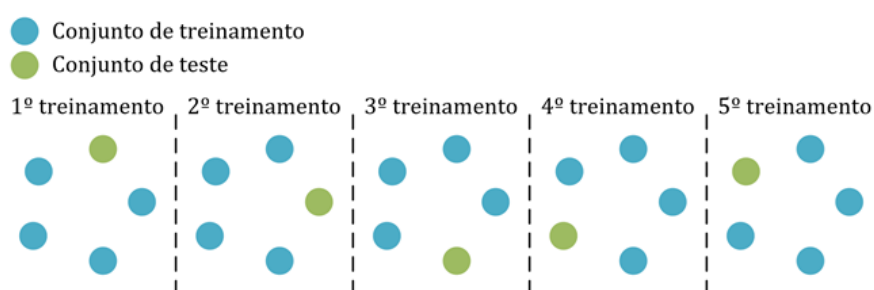


Figura 21 – Divisão do conjunto de amostras pelo método de validação cruzada *k-fold*

Durante a fase de treinamento, é necessário ainda estipular quantos neurônios serão inseridos nas camadas ocultas da rede PMC. O número de neurônios e de camadas pode ser definido de maneira empírica, testando várias combinações de número de camadas e de neurônios em cada camada. É importante salientar que a rede PMC com uma camada escondida já é suficiente para resolver muitos problemas com não linearidades, como classificação de dados não linearmente separáveis e modelagem de funções não lineares.

Se voltarmos ao exemplo das amostras com classes não linearmente separáveis da Figura 11(b), é possível fazer a separação das classes com uma rede perceptron multicamadas. A Figura 22(a) mostra uma topologia de rede PMC que consegue separar as amostras não linearmente separáveis. Os neurônios O_1 e O_2 são os responsáveis por fazer cortes em formato de retas no espaço de amostras da Figura 22(b). Todas as amostras que estão à esquerda da reta representada pelo neurônio O_1 e abaixo da reta do neurônio O_2 são rotuladas como Classe B. As outras amostras são rotuladas como Classe A. O neurônio S_1 recebe as informações da camada oculta e rotula as amostras como Classe A ou Classe B.

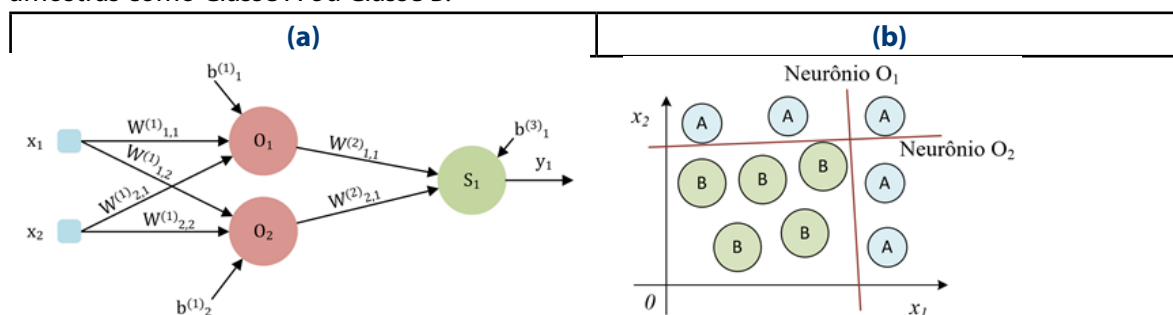


Figura 22 – Topologia de (a) rede perceptron multicamadas que pode classificar as amostras não linearmente separáveis e (b) classificação de amostras não linearmente separáveis

A topologia que consegue classificar as amostras, ou fazer previsões precisas para outros problemas, pode ser encontrada de maneira empírica, como já mencionado anteriormente. Uma

dessas maneiras é utilizar o método de validação cruzada *k-fold* e encontrar a topologia com o menor erro quadrático médio de todos os treinamentos de cada partição. Entretanto, é necessário ficar atento quanto à capacidade de generalização do modelo e evitar o fenômeno de *overfitting* e *underfitting*.

Durante a fase de treinamento de uma rede PMC, o objetivo do ajuste dos parâmetros do modelo é fazer previsões precisas e reduzir ao máximo o erro quadrático médio encontrado a cada rodada do algoritmo de aprendizado. Isso é feito ajustando os parâmetros de cada neurônio da rede, aumentando o número de neurônio da camada oculta ou, ainda, o número de camadas ocultas. Porém, muitas vezes, o aumento do número de neurônios ou de camadas faz com que a rede PMC decore todos as amostras do conjunto de treinamento, tornando o modelo muito especialista, mas somente para os pontos que ele conhece.

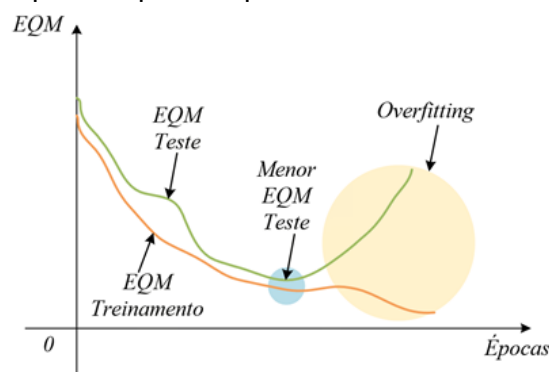


Figura 23 – Identificação da região de *overfitting* durante o treinamento da rede PMC



Uma maneira de identificar se está ocorrendo o *overfitting* é verificar o erro quadrático médio (EQM) da fase de treinamento e da fase de teste do modelo. A Figura 23 apresenta um caso típico da ocorrência de *overfitting*, que o EQM do treinamento continuou diminuindo com o aumento do número de épocas de treinamento, mas o EQM de teste foi aumentando. O melhor momento para encerrar o treinamento, nesse caso, era quando o EQM de teste atingiu o menor valor da curva e o EQM de treinamento estava baixo também. No fenômeno de *underfitting* acontece o contrário e a rede PMC não é capaz de inferir respostas precisas durante a fase de teste nem durante a fase de treinamento. Nesse caso, um aumento no número de neurônios pode resolver o problema de identificar características importantes nas amostras do conjunto de treinamento.

Os algoritmos de busca paramétrica estão sempre sujeitos a buscas que terminam com um conjunto de parâmetros subótimos, também denominado ponto de mínimo local (Figura 24). O algoritmo de gradiente descendente identifica que não é mais necessário alterar os valores dos parâmetros da rede PMC e encerra a fase de treinamento do modelo. O seguimento de um caminho que leva ao mínimo global ou ao mínimo local em um algoritmo de *backpropagation* utilizando o gradiente descendente depende das condições iniciais do problema. No caso da Figura 24, o vetor (conjunto de pesos sinápticos e limiares de ativação de todos os neurônios da rede) foi inicializado no lado direito da curva. Se por acaso ele tivesse sido inicializado do lado esquerdo, o algoritmo teria convergido para o ponto de mínimo global. Portanto, é necessário que haja outras ferramentas para aumentar as chances de não cair em um mínimo local ou de seguir o caminho até atingir o mínimo global.

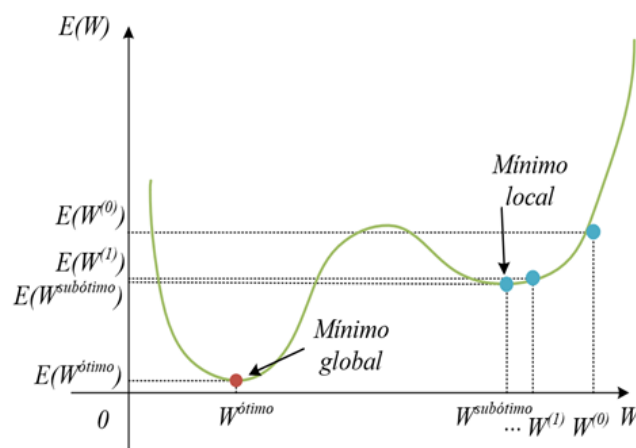


Figura 24 – Topografia de erro com mínimo local e mínimo global.



Coordenadoria de
Educação Aberta e a Distância